Systolic Array Matrix Multiplication Accelerator for PicoRV32 on FPGA

Aditi Dure (EE23BTECH11016)
Krishna Kaustubh (EE23BTECH11005)

1. Summary

This Stage-2: Implementation-1 report presents progress on the accelerator proposed in Stage-1. The Verilog design now includes a systolic array, controller, memory-mapped interface, and PicoRV32 integration. Bare-metal C firmware compares accelerator results and execution cycles with a triple-nested-loop software implementation. Verification has been performed in simulation; FPGA implementation remains pending.

2. Design Approach

Processing Element and Systolic Array

The design uses an output-stationary 4×4 mesh of sixteen processing elements (PEs) to compute

$$C_{i,j} = \sum_{k=0}^{K-1} A_{i,k} B_{k,j}.$$

A operands move rightward and B operands move downward, while partial sums remain inside each PE. Input lanes are delayed by 0, 1, 2, and 3 enabled cycles to align corresponding operands. Independent valid bits control operand acceptance.

Each PE uses radix-4 Booth recoding and a carry-save compressor tree. The accumulator retains sum and carry vectors, with final carry-propagate addition performed when a drain marker publishes the result. Inputs are signed INT8, and internal accumulation is 24 bits, sufficient for the supported maximum inner dimension $K = 256$. The array accepts $4 \times K$ and $K \times 4$ inputs; its output dimensions remain 4×4.

Controller and Memory-Mapped Integration

The register-backed controller performs one 4×4 multiplication per accepted start request. Its FSM clears the array, feeds four operand steps, sends a drain marker, collects all sixteen results, and asserts completion. A received-result mask determines completion. Input writes and additional start requests are ignored while busy.

The MMIO base address is `0x10000000`. PicoRV32 firmware writes the inputs, starts computation, polls status, and reads the sign-extended results. A request adapter provides the gap between peripheral accesses. The system testbench supplies program and data RAM.

3. Tools and Frameworks Used

- **Verilog-2001 and PicoRV32:** Accelerator RTL and processor integration.
- **Icarus Verilog:** Compilation and self-checking simulation.
- **Python :** Exact golden model, test vectors, and verification scripts.
- **RISC-V GCC and binutils:** Bare-metal RV32IM firmware compilation.
- **GNU Make and GTKWave:** Test automation and waveform inspection.
- **Supporting tooling:** Verilator lint flow and a Yosys/ELK.js RTL viewer.

4. Implementation Status and Verification

The arithmetic modules, clocked PE, fixed array, controller, MMIO interface, and processor firmware are implemented. Self-checking testbenches use Python-generated expected results. Available coverage includes signed extremes, zero and identity matrices, deterministic random inputs, continuous streams, mixed inner dimensions, and control/reset behavior.

Preliminary Cycle Comparison

For each job in the default processor simulation, the sequential software multiplication took 5388 cycles and the complete accelerator driver call took 1422 cycles, giving a cycle ratio of approximately 3.79. The driver measurement includes input transfers, start, polling, and result reads. Both measurements include call/counter overhead and exclude output checking and reporting. These results depend on compiler and simulated-memory settings; FPGA timing and an optimized CPU baseline have not been evaluated.

5. Changes from the Stage-1 Plan

- **Array parameterization:** The current mesh is fixed at 4×4; general $N \times N$ support is not yet implemented. Variable K changes the inner dimension, not the mesh size.
- **Accumulation width:** Stage-1 proposed INT32 accumulation. The current datapath uses 24-bit accumulation, with results sign-extended to 32 bits at the processor interface. This width supports the current INT8/$K \leq 256$ range but is a change from the proposed internal width.
- **Matrix buffers:** The proposed BRAM buffers are currently implemented as register banks. These provide four A and four B reads per feed step; a BRAM implementation requires a compatible banking/read schedule.

The memory-mapped integration, golden-model verification, and C cycle-comparison flow follow the original approach. The Booth/carry-save architecture provides an implementation detail for the PE, which was not specified in Stage-1.

6. Pending Tasks

- Implement array-size parameterization and BRAM buffering; resolve whether to restore INT32 accumulation or retain the documented width change.
- Select the FPGA board and add the top-level memory, clock/reset, pin assignments, and timing constraints.
- Run synthesis and implementation, report resource utilization and achieved timing, and demonstrate correct operation on hardware.
- Complete regression/lint checks and measure on-board end-to-end performance against the software baseline.
- Pursue the original stretch goals after the must-have deliverables: PCPI custom instructions, larger-matrix support, and a 4×4 versus 8×8 scaling study.

7. Disclosure of LLM Assistance

LLM assistance was taken to write the automation and the scripts and golden model in python for checks and comparisons. The verilog code and testbenches were assisted by AI merely for betterment of the code.